

Compliance Document Review App

Intern Project Brief

Program: Capture the Flag intern challenge (Project phase)

Timebox: 1 month

Team: Interns split across Backend / Frontend / AI

The problem

Financial advisors produce client-facing material — marketing emails, brochures, social posts, meeting notes, proposal letters — and none of it can go out until a compliance officer has reviewed it. Today that hand-off is ad hoc: there is no shared place to submit a document, no visible queue for the reviewer, no record of what was decided or why, and the reviewer has to read every file end to end before they can even tell what it is.

Build a web app that closes that loop.

What it has to do

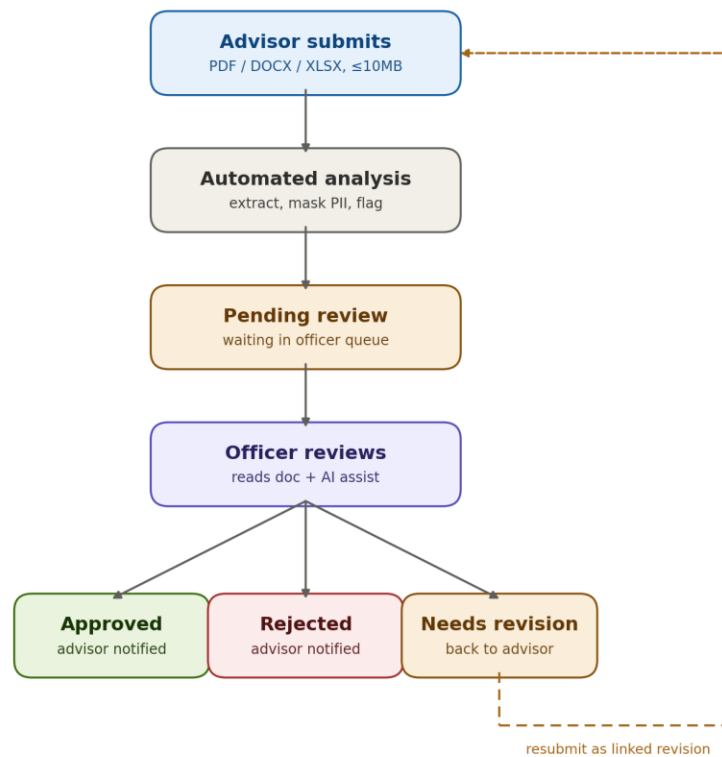
An Advisor signs up, submits a document, and later sees its status and the feedback it received.

A Compliance Officer signs up, sees every advisor's submissions in one queue, opens a document, and records a decision plus a comment. Alongside the document they get an AI assist — a short summary of the file and a list of potential compliance issues (missing disclosures, inconsistent figures or dates, guaranteed-return or unapproved performance claims, missing signature or approval fields). The assist orients the reviewer; the human decision is the one that counts.

A document moves from Pending Review to Approved, Rejected, or Needs Revision. Needs Revision sends it back to the advisor, who submits a revision — and that revision is linked to what it replaces, so both sides can read the document's history as one thread rather than as unrelated uploads.

The document lifecycle

Every submission moves through the same path. The assist and audit trail run alongside it — analysis is generated once and cached, and each state change is recorded.



Colour key: blue = advisor action, purple = officer action, grey = the automated step, amber = states waiting on someone, green / red = final outcomes. Needs Revision loops back and the resubmission stays linked to the original, so the history reads as one thread.

Team and responsibilities

Five tracks build the app together. The split below is the natural division of the work — the integration points (masking, the vector store, rendering the AI output, the setup that ties it all together) cross tracks, so pairing across boundaries is expected rather than a clean hand-off.

Track	Owns
Backend	Sign-up with two fixed roles; session-gated access to all document data; role enforcement on the server (an advisor hitting an officer endpoint is rejected, not just hidden). File upload (PDF / DOCX / XLSX, 10MB cap), the document state machine, revision linking, the audit trail, and in-app notifications.
Frontend	Both dashboards: the advisor's submit-and-track view and the officer's status-filterable queue with the decision-plus-comment action. The AI assist panel — summary, flags with their reasoning, precedent matches —

Track	Owns
	plus the error, empty, and degraded states so the review page still works when the AI is unavailable.
AI	The server-side PII masker and the summary-and-flags generation against a free-tier third-party API. Each flag carries its passage, its rule, and a one-line reason. Works with Data engineering on what gets retrieved and how the model checks the document against it.
Data engineering	The ingestion-and-retrieval pipeline: extracting clean text from PDF / DOCX / XLSX for the masker; chunking, embedding the masked text, storing the vectors, and the similarity queries behind all three retrieval jobs (rule lookup, disclosure-by-absence, precedent); and tuning retrieval quality — chunk size, what to embed, the distance metric, and the present-vs-missing threshold that decides whether a disclosure counts as there. Owns whether retrieval actually finds the right rule. (The precedent count is fixed at three by the spec.)
DevOps / platform	Reproducible setup and delivery. The spec fixes the outcome, not the tooling: anyone can clone, follow the README, and reach a running app, and the test suite passes from a clean checkout. The usual means — the team's call, not mandated — are a container setup (e.g. docker-compose) bringing up the app plus Postgres / pgvector, the API key handled through env vars and an example file (never committed), and database setup plus corpus seeding as scripted, repeatable steps, with CI running the tests on each push as a recommended add. Owns the clean-checkout path the whole team is graded on.

Shared by all tracks. Tests covering at least the role boundary and the masker; a clean-checkout setup with a documented README; no real client data anywhere; no production Anthropic credits spent on the AI; and, as a one-time job of about half a day, generating the seed corpora — the rules, disclosures, and roughly 100 sample documents — with an LLM.

Constraints

- Two roles, fixed at sign-up. Advisors cannot review; officers cannot submit.
- Role enforcement lives on the backend, not just in the UI — an advisor hitting an officer endpoint is rejected, not merely hidden from. All document data requires a signed-in session.
- Any officer can act on any document; there is no per-officer routing.
- Uploads: PDF / DOCX / XLSX, 10MB cap.
- Notifications are in-app only — the advisor sees the update next time they open their dashboard.

- Every state change is recorded and readable: who submitted, who viewed, who decided what and when. A decision nobody can trace back is worth very little.
- The build is covered by tests and runs from a clean checkout with documented setup. Over a month, “it works on my machine at the demo” is not a pass.
- Tech stack is otherwise the team's call. The rubric scores architecture and quality decisions, not stack choice.

AI constraints

The AI feature runs against a third-party free-tier API (Gemini via Google AI Studio recommended; Groq or OpenRouter also fine — do not spend production Anthropic credits). That single fact drives most of what follows: document text is leaving your perimeter to a vendor whose free tier may train on it.

- **Mask PII before the document ever leaves the app.** Names, emails, phone numbers, addresses, account and SSN-style numbers, and dollar amounts tied to a named person get replaced with stable placeholders ([CLIENT_1], [ACCOUNT_1]) on the way out, and the mapping stays server-side. The model reasons over masked text; the officer reads the original. Anything the model returns that references a placeholder is unmasked for display only. A submission that reaches the vendor unmasked is a failed requirement, not a rough edge.
- **Masking is a server-side step, not a prompt instruction.** “Please ignore personal data” is not masking. It happens in your code, before the API call, and you should be able to show the exact payload that was sent.
- **Nothing gets embedded unmasked either.** Embeddings are an API call too. If you send raw text to a hosted embedding model you have leaked exactly what the masker was there to prevent, just through a different endpoint. Mask first, embed the masked text — similarity search works fine on it.
- **Every flag shows its reasoning.** A flag is not a verdict — it carries the passage it fired on, the rule it retrieved, and a one-line explanation of why the two conflict. An officer must be able to disagree with a flag and see exactly where it went wrong. Bare severity labels with no trace are not acceptable output.
- **The AI never decides.** It cannot set, suggest-and-apply, or pre-fill a review status. Only a compliance officer writes a decision.
- **Cache the summary and flags per document.** Analysis runs once on submission (or first open), not on every page load. Free tiers are rate-limited and the officer shouldn't wait on a network call to read a queue item.
- **Degrade gracefully.** If the API is down, rate-limited, or returns garbage, the review UI still loads and the officer can still work — the assist panel shows an error state and a retry, and the document does not get stuck.

Vector search

A vector store is required, and it earns its place three ways. Don't ask a general-purpose model “is this compliant?” — retrieve first, then have it check the document against what came back.

1. Rule retrieval. Embed a corpus of compliance rules — required disclosures, prohibited claim language, performance-claim standards. For each section of a submitted document, pull the rules that actually apply and check against those. Keeps prompts small and every flag traceable to a rule.

2. Missing-disclosure detection by absence. Embed the firm's approved disclosure and disclaimer texts. A required disclosure is present if some passage in the document sits close to it in vector space — reworded, reformatted, it still matches. Nothing nearby means it's missing. Keyword matching cannot do this, because disclosures get paraphrased, and it detects absence rather than presence.

3. Precedent search. Embed every reviewed document alongside its decision and the officer's comment. When a new submission lands, show the officer the three most similar past documents and how they were decided. Compliance runs on precedent, and two officers reaching opposite conclusions on near-identical documents is a real failure this directly prevents.

Seed the corpora properly — a dozen rules and three documents will make any vector store look pointless. Aim for a few dozen rules and disclosures and a hundred-odd seeded documents so retrieval quality becomes something you can actually feel and tune. Generating the corpora with an LLM is fine and expected — budget half a day for it. The rules can be plausible inventions modelled on real advertising and disclosure standards; nobody is grading regulatory accuracy, only whether retrieval finds the right rule and the flag explains itself. Use no real client data of any kind.

Stretch. Semantic diffing of revisions — embed old and new versions chunk-wise so the officer sees which paragraphs actually changed. Also: clustering near-identical submissions across advisors, so the same templated newsletter sent by twelve people is reviewed once.

On tooling. pgvector is the recommended default if there's a Postgres already — the lesson that vector search is an index next to your data rather than a separate product is worth learning. Chroma or Qdrant are fine. Avoid anything whose free tier expires mid-project.

Data model — a starting sketch

A shape to react to, not a prescribed schema. Field names and structure are yours to decide — refining this, and justifying your choices, is part of what's assessed.

Entity	Key fields and links
User	id, role (advisor officer), name, credentials. Role is fixed at sign-up.
Document (submission)	id, advisor_id, status, file reference, type, uploaded_at, link to the thread it belongs to.
Revision link	each resubmission references the document it replaces, forming an ordered thread.

Entity	Key fields and links
Review / decision	id, document_id, officer_id, status (approved rejected needs_revision), comment, decided_at.
AI analysis (cached)	document_id, summary, generated_at. One per document; regenerated only on retry.
Flag	id, analysis_id, passage excerpt, matched_rule_id, explanation, severity.
Audit event	id, actor_id, document_id, action (submitted viewed decided resubmitted), timestamp. Append-only.
PII mapping	document_id, placeholder to original value. Server-side only, never sent to the vendor.
Rule / disclosure	id, text, type, embedding vector. The retrieval corpus.
Precedent index	masked reviewed-document text + decision + comment, with an embedding vector, for similarity search.

Shape of the month

Not a mandated plan, but the failure mode to avoid is spending three weeks on the AI and demoing an app that can't sign a user in. Get the boring half working first.

Week 1 — Auth, roles, backend permission enforcement, upload, the two dashboards. End the week with a document that can be submitted and decided on, no AI involved.

Week 2 — The AI path end to end: text extraction, PII masking, the rule corpus, retrieval, summary and flags rendered in the review panel.

Week 3 — The parts that make it a compliance tool rather than a form: disclosure-by-absence detection, precedent search, revision threads, audit trail, filtering, error and empty states, graceful degradation when the API is gone.

Week 4 — Tests, clean-checkout setup, and hardening. Try to break your own role boundary and your own masker, then fix what breaks.

Getting started

- **Agree the stack as a team.** Recommended default: Postgres with pgvector, so the vector store sits next to your data. Chroma or Qdrant are fine alternatives.
- **Get a free-tier LLM API key.** Gemini via Google AI Studio (recommended), or Groq / OpenRouter. Do not use production Anthropic credits.
- **Clone and run setup from the README.** The clean-checkout path is a graded requirement, so keep it working from day one rather than fixing it the night before the demo.
- **Build the boring half first.** Auth with two fixed roles, then upload and the two dashboards. Get a document submitted and decided before adding any AI.

- **Seed the corpora.** A few dozen rules and disclosures and roughly 100 seeded documents. Generating them with an LLM is expected — budget about half a day. Use no real client data.
- **Sanity-check the privacy wall early.** Submit a document with a fake client name, email, and account number, then inspect the exact outbound payload and confirm the real values are gone.

How you'll be assessed

The rubric rewards a working end-to-end tool with the role boundary and the privacy wall genuinely holding. Concretely, these are the things that get checked:

What we check	Passing looks like
Functional loop	Advisor submits a real file; officer finds it in a status-filtered queue, decides with a comment; advisor sees the decision.
Role boundary	Both directions hold when probed at the API, not just hidden in the UI.
PII masking	The payload sent to the vendor carries placeholders, not the real client name, email, or account number — and masking happens in code, provably.
Traceable flags	Each flag points to its passage and the rule it was checked against, with a one-line reason.
Graceful degradation	With the API key removed, the review page still loads and still accepts a decision.
Revision & audit	A revise-resubmit-approve cycle reads as one linked thread; the audit trail shows who did each step and when.
Tests & setup	Suite passes and covers at least the role boundary and the masker; someone outside the team can clone, follow the README, and reach a running app.
Architecture & quality	The rubric scores design and quality decisions, not stack choice.

Out of scope

Per-officer assignment and routing — any officer can act on any document. Skip it unless everything else is genuinely finished.

On the AI side: no fine-tuning or training anything, no self-hosting a model, and no production-grade PII detector. A careful regex-and-heuristics masker over the common entity types — with a test suite covering the cases it's meant to catch and honest notes on what it misses — is the expected depth. Knowing where your masker fails counts for more here than an extra week of pattern-writing.

Done means

An advisor can sign up, submit a real file, and see the officer's decision and comment on it. An officer can sign up, find that submission in a queue they can filter by status, read an AI summary and flags for it, and record their decision. Both directions of the role boundary hold when probed at the API, not just in the browser. A document sent back for revision, resubmitted, and approved reads as one thread with its full history, and the audit trail shows who did each step and when.

On the AI side: submit a document seeded with a fake client name, email, and account number, then show the exact payload sent to the vendor — the real values are absent and placeholders are in their place. Each flag on that document points at the passage that triggered it and the rule it was checked against. Kill the API key and the review page still loads and still accepts a decision.

Someone outside the team can clone the repo, follow the README, and reach a running app. The test suite passes and covers at least the role boundary and the masker.